

```
]
print("Строк в срезе:", len(subset))
print(subset.head())
```

Результат выполнения

Строк в срезе: 337

Таблица 9. Результат выполнения блока 7.3 как таблица; результат показывает первые 5 наблюдений из подвыборки

type_name	lon	lat	temperature_c	relative_humidity	wind_speed_ms
Forest fire	91.39	51.06	15.15	27.97	2.55
Forest fire	104.58	50.37	14.67	37.89	2.64
Forest fire	104.49	50.36	14.67	37.89	2.64
Forest fire	104.42	50.33	14.67	37.89	2.64
Forest fire	104.30	50.34	14.67	37.89	2.64

Условия `relative_humidity < 40` и `wind_speed_ms > 2` выбраны только как учебный пример составного фильтра, а не как нормативный критерий пожарной опасности. В наборе им соответствуют 337 строк.

8. Корреляция и потенциальные выбросы

Коэффициент корреляции Пирсона оценивает линейную связь двух числовых признаков и лежит в диапазоне от -1 до 1 . Значение около нуля означает отсутствие выраженной линейной связи, но не отсутствие любой зависимости. Корреляция также не устанавливает причинность.

Для первичного поиска необычных значений можно использовать межквартильный размах $IQR = Q3 - Q1$. Эвристическое правило помечает значения за пределами $[Q1 - 1,5 \cdot IQR; Q3 + 1,5 \cdot IQR]$. Такие наблюдения являются кандидатами на дополнительную проверку, а не автоматически ошибками.

Практический блок 8.1. Корреляционная матрица

Код данного блока построит матрицу попарных линейных связей, после чего нужно анализировать знак и модуль коэффициентов. Цель — определить, какие пары признаков демонстрируют заметную линейную сопряжённость, не превращая корреляцию в утверждение о причинно-следственной зависимости.

В расчёт сознательно включаются только метеорологические показатели: категориальный код `type_id` и географические координаты здесь исключены из рассматриваемой содержательной задачи.

Список `weather` фиксирует набор переменных, между которыми имеет смысл оценивать линейные связи. Выражение `df[weather]` создаёт числовой поднабор `DataFrame`.

Метод `corr()` по умолчанию рассчитывает попарные коэффициенты корреляции Пирсона и возвращает квадратную симметричную матрицу. На диагонали находятся единицы, поскольку каждый признак полностью коррелирует сам с собой. `round(3)` ограничивает вывод тремя знаками после десятичной точки.

```
weather = [
    "temperature_c", "precipitation_mm", "relative_humidity",
    "wind_speed_ms", "solar_radiation"
]

corr = df[weather].corr()
print(corr.round(3))
```

Таблица 10. Корреляционная матрица для атрибутов набора данных

Признак	temperature_c	precipitation_mm	relative_humidity	wind_speed_ms	solar_radiation
temperature_c	1.00	0.00	-0.15	-0.30	0.22
precipitation_mm	0.00	1.00	0.23	-0.05	-0.19
relative_humidity	-0.15	0.23	1.00	-0.28	-0.23
wind_speed_ms	-0.30	-0.05	-0.28	1.00	-0.19
solar_radiation	0.22	-0.19	-0.23	-0.19	1.00

В данном наборе нет очень сильных линейных связей между рассмотренными погодными признаками. Наибольшие по модулю значения среди пар находятся примерно на уровне 0,30: temperature_c и wind_speed_ms имеют $r \approx -0,297$, relative_humidity и wind_speed_ms — $r \approx -0,280$. Это умеренно слабые связи и не основание для причинных выводов.

Практический блок 8.2. Правило IQR на примере скорости ветра

Следующий фрагмент не удаляет данные, а только маркирует необычные значения скорости ветра по стандартному правилу $1,5 \cdot \text{IQR}$. Сначала вычисляются квартильные характеристики и границы, затем строится булева маска всех наблюдений, лежащих вне интервала. В конце выводятся сами параметры правила и число кандидатов.

Метод `quantile(p)` возвращает квантиль заданного уровня. Квантиль уровня $p=0.25$ соответствует первому квартилю $Q1$, $p=0.75$ — третьему квартилю $Q3$. Межквартильный размах вычисляется как $iqr = q3 - q1$, после чего формируются эвристические границы $Q1 - 1.5 \cdot \text{IQR}$ и $Q3 + 1.5 \cdot \text{IQR}$.

Два сравнения с границами объединяются оператором `|` — поэлементным логическим «ИЛИ». Полученная маска содержит `True` для кандидатов на выброс, а `mask.sum()` считает их число.

```
q1 = df["wind_speed_ms"].quantile(0.25)
q3 = df["wind_speed_ms"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

mask = (
    (df["wind_speed_ms"] < lower) |
    (df["wind_speed_ms"] > upper)
)

print(f"Q1={q1:.3f}; Q3={q3:.3f}; IQR={iqr:.3f}")
print(f"Границы: [{lower:.3f}; {upper:.3f}]")
print("Кандидатов:", mask.sum())
```

Результат выполнения

```
Q1=0.110; Q3=0.660; IQR=0.550
Границы: [-0.715; 1.485]
Кандидатов: 3652 (13.69% строк)
```

Правило IQR помечает 3 652 наблюдения по скорости ветра. Такая доля слишком велика, чтобы автоматически считать эти строки ошибочными: скорее распределение имеет выраженный правый хвост. В реальном проекте решение об удалении возможно только после проверки источника и предметной интерпретации.

Такой порядок выполнения операций подчёркивает, что статистическая эвристика создаёт список для проверки, но не принимает решение об ошибочности наблюдений.

9. Визуальный разведочный анализ

График должен отвечать на конкретный аналитический вопрос. Столбчатая диаграмма удобна для сравнения категорий, гистограмма — для формы распределения одной числовой величины, график разброса — для взаимосвязи двух числовых переменных, временной ряд — для значения, упорядоченного по времени. Выбор типа графика определяется структурой данных, а не желанием использовать как можно больше визуализаций.

В таблице нет даты или времени события. Поэтому построение временного ряда «число пожаров по месяцам» на основе этого файла невозможно. Создавать искусственную дату или использовать индекс строки как время методологически неправильно.

Практический блок 9.1. Bar chart частоты типов событий

Перед построением графика частоты будут рассчитаны заново и отсортированы так, чтобы горизонтальные столбцы удобно сравнивались визуально. Полученная диаграмма предназначена не для поиска точных чисел — они уже известны из таблицы частот, — а для быстрой оценки относительных масштабов категорий и дисбаланса классов. Заголовок и подписи осей добавляются как обязательная часть аналитической визуализации.

Импорт библиотеки `import matplotlib.pyplot as plt` подключает модуль `pyplot` библиотеки `Matplotlib` под коротким именем `plt`.

Метод `value_counts()` вычисляет частоты категорий, `sort_values()` упорядочивает их, а `Series.plot(kind="barh")` использует интеграцию `Pandas` с `Matplotlib` для построения горизонтальной столбчатой диаграммы. Параметр `figsize` задаёт размер рисунка.